Ask a Language Model How Good the Model Is Before It Answers Anything

Siwei Shan[*] NLPQuant AI Inc.
Sunny Yang[*] University of Illinois Urbana-Champaign guangy4@illinois.edu
Weiyan Zhao NLPQuant AI Inc.

Abstract

How much evidence does it take to predict how an AI system will answer a benchmark it has never seen? Most of the error comes before the system has answered anything, when the predictor does not know how strong it is. A language model already knows much of this. Given a system's name and a few of the benchmark's items, Claude Opus estimates the system's accuracy about as well as six of its own answers would. Without the name, the estimate is barely better than a guess. We use this estimate as the starting point of a predictor that pools answers across systems. In the competition's live runs it cut the error before the first answer by about three quarters. Code, cached model outputs and the simulator are released.

1 Introduction

The Predictive AI Evaluation Competition asks how likely a subject (a model under a given configuration) is to answer each item of an unseen benchmark correctly. Predictions are scored after the subject reveals 0, 1, 3, 7, 15 and 31 of its own answers, by the Brier area under that learning curve (ALC). Budgets 0 to 3 carry half of the weight, and that is where predictors lose most. With no answers yet, a predictor cannot tell a model that answers nearly everything from one that answers half.

Our idea is to ask a language model. It has read about most named models and can judge how hard the items look, so it can say how often a given model will succeed before the model answers anything. We turn its estimate into a prior that the model's own answers then update. Our contributions:

- **The finding** (Section 4). Claude Opus estimates a model's accuracy on a new benchmark as well as about six of the model's own answers would. The value comes from knowing which model it is.
- **The effect on prediction** (Sections 5 and 6). On identical simulated draws the prior is the largest single gain of our predictor, with one or four subjects per benchmark. In the live runs it removed about three quarters of the error before the first answer.
- **A reusable kit**: the predictor, a simulator of the competition protocol, and every cached language-model output, so the simulations rerun without an API key.

Related work. The organisers' Bayesian Linguistic Evaluator also starts from a language model's estimate, but it asks for every target response separately and revises the estimate in the prompt. We ask once per subject and benchmark and update the estimate numerically. Language models have also been asked to forecast benchmark scores from task descriptions [9], and missing scores can be completed from a model's other scores [10]. We use such a forecast as the prior of an item-level predictor. Methods that predict from few items usually assume items already calibrated on many

[*]Equal contribution.

Technical report, Predictive AI Evaluation Competition (PAIEC) 2026.

Table 1: How well a language model estimates a subject's accuracy before any answers, on 71 held-out subject–benchmark pairs from public benchmarks. "Answers" is the number of the subject's own answers with the same mean squared error.

Estimate	RMSE	Correlation	Answers
Constant 0.5	0.217	–	0
Claude Haiku 5.5	0.185	0.63	3
Claude Sonnet 5.5	0.142	0.76	7
Claude Opus 5.5	0.141	0.76	7
Opus, subject's identity hidden	0.211	0.28	1
Opus, benchmark named in the prompt	0.136	0.78	8

models [1–4], which a new benchmark lacks. In measurement terms, the task is a Rasch model [5, 6] fitted online from very few responses, as in adaptive testing [7], and our prior is the collateral information that sets an examinee's starting ability [8].

2 Task

For each subject–benchmark pair, items are split 50/50 into an acquisition pool and an evaluation pool. Acquisition items stream past, and the program decides which to reveal, up to 31. Predictions on the evaluation pool are scored at budgets $b \in \{0, 1, 3, 7, 15, 31\}$ with weights $0.1, 0.2, 0.2, 0.2, 0.2, 0.1$, and the ALC is averaged over pairs. Revealed labels are shared across all pairs in a run. The program sees item text and subject attributes, never reference answers.

3 Method

Subject prior. We ask Claude Opus 5.5 what share of the benchmark's items the subject answers correctly, given up to eight sample items and the subject's name, provider, release date, harness and reasoning effort. Eight solved examples from public benchmarks precede the question and fix the scale. The estimate q_s is shrunk toward one half, $m_s = 0.5 + 0.75\,(q_s - 0.5)$, used directly at budget 0, and then acts as a Beta prior with four pseudo-counts:

$$\hat{\mu}_s = \frac{\sum_{j \in R_s} y_{sj} + 4\, m_s}{|R_s| + 4}, \tag{1}$$

where R_s is the set of items s has revealed.

The rest of the predictor. When other subjects share the benchmark, $\hat{\mu}_s$ is also pulled toward their success rate. Each item's prediction then moves with three signals: the answers other subjects gave to the same item (one Newton step of a Rasch model), the residuals of similar revealed items, and a difficulty rating from Claude Opus calibrated on the revealed labels. The acquisition rule reveals items no other subject has revealed yet. Appendix C tests these parts on their own.

Running on the platform. The platform recreates its worker processes at every budget. The predictor therefore stores every language-model result on disk for all workers to share, and caps calls per worker. Without an API key it runs on labels alone. One formative run costs $10 to $15.

4 The language model knows the model

We measured the prior on public benchmarks, against each subject's true accuracy. To state its error in the competition's currency, we find the number of the subject's own answers n at which a Laplace estimate $(k + 1)/(n + 2)$ has the same mean squared error.

Table 1 shows the result. Opus and Sonnet estimate accuracy as well as about seven of the subject's own answers, and Haiku as well as three. With the fallback the submission uses and the shrinkage chosen on other benchmarks, the deployed prior is worth six answers (Appendix A). Simpler priors

Table 2: The submitted predictor on 90 identical simulated draws. Differences are paired over draws, against the variant with both language-model parts off.

Variant	ALC	B_0	Difference	Better in
Laplace estimate on own answers	0.2020	0.250	$+0.0008 \pm 0.0002$	20% of draws
Neither part	0.2012	0.250	–	–
Item ratings only	0.1992	0.244	-0.0020 ± 0.0004	68% of draws
Subject prior only	0.1941	0.221	-0.0071 ± 0.0009	80% of draws
Both (submitted)	**0.1924**	0.216	-0.0088 ± 0.0010	83% of draws

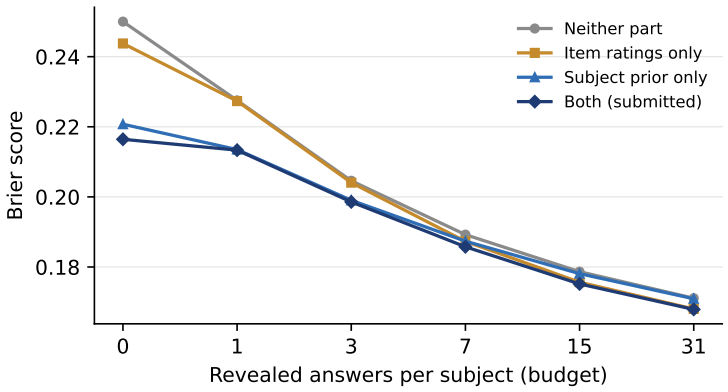


Figure 1: Brier score by budget on the 90 simulated draws of Table 2. The subject prior closes most of the gap at budgets 0 to 3.

do not work. A subject's accuracy on other benchmarks predicts its accuracy on a new one no better than a constant one half.

The bottom rows show where the knowledge comes from. When we hide the subject's name, provider and release date, the estimate falls almost to the constant. When we name the benchmark, which the deployed prompt never does, it barely changes. What the language model contributes is its knowledge of the model.

5 The prior improves the predictor

To isolate the prior, we ran the submitted predictor on identical simulated draws in the competition's online setting, nine pairs over seven benchmarks with mostly one subject each, and switched its language-model parts off in turn. The subject priors are real Opus calls with the deployed prompt, cached so every variant sees the same estimates.

Table 2 and Figure 1 give the result. With one subject per benchmark, the predictor without its language-model parts is barely better than a smoothed average of the subject's own answers. The prior is the largest part of the gain. It lowers the error at budget 0 from 0.244 to 0.216, and its effect fades by budget 7, once the subject's own answers take over. With four subjects on every benchmark, where pooling works from the first answer, the prior helps more consistently: -0.0076 ± 0.0008, better in 93% of 30 draws.

6 Live results

Each formative run draws nine subject–benchmark pairs from seven benchmarks, and in our runs most benchmarks had a single subject. Raw scores swing with the draw. Three runs of the same final package scored 0.130, 0.181 and 0.164. We therefore look at each pair's *excess*, the Brier score at each budget minus the pair's score at budget 31, when the predictor knows the subject well. It measures how fast the predictor learns the subject.

Table 3: Formative runs on Codabench. Excess is the Brier score above each pair's budget-31 level, averaged over pairs. Submission IDs and per-run results are in Appendix E.

Version	Runs	$b = 0$	$b = 1$	$b = 3$	$b = 7$	Weighted
Pooled baseline	3	0.119	0.048	0.023	0.018	0.031
+ subject prior	5	0.026	0.018	0.008	0.009	0.011
+ Opus item ratings (final, 1017682)	3	0.038	0.025	0.003	-0.004	0.008

Table 3 shows the prior at work. Before any answers, the baseline's excess was 0.119: it guessed one half for subjects that answered almost everything. The prior cut that to about 0.03, and the weighted excess from 0.031 to 0.011. Online the prior was worth two to three answers rather than six, because many live subjects answered nearly every item, and then a few answers already pin them down.

A lesson for the platform. Because workers are recreated at every budget, a result kept in memory reaches only the worker that asked for it. Our first deployment gained a quarter of what the prior delivers once results were shared on disk. A scratch directory shared by the workers of one evaluation would let language-model submissions call each model once, cutting cost and run-to-run noise.

7 Limitations

The prior depends on the language model knowing the subject. For unnamed subjects the predictor leaves it out. On 15 benchmarks never used in development it made no measurable difference: -0.0011 on those with named subjects (90% interval -0.009 to $+0.007$), helping on five and hurting on three (Appendix B). The simulation benchmarks also served for development, so Table 2 may be optimistic. Online subjects were stronger than those in the training data, and the shrinkage toward one half makes the prior too cautious for models that answer nearly everything, one reason it was worth fewer answers there. When two subjects of very different strength share a benchmark, pooling pulls the weaker one toward the stronger, which hurt a few online pairs at budgets 15 and 31. The live comparison spans different draws, and we did not compare against the organisers' Bayesian Linguistic Evaluator, whose default configuration failed on the current public data. The method needs a paid API key, and all benchmarks we used have binary responses.

8 Reproducibility and disclosures

Code, the simulator, cached language-model outputs and the logs behind every simulation are at `https://github.com/sunnyadn/paiec-pooled-priors`. `python reproduce.py` prints the simulation tables from the logs. With access to the gated measurement-db, the documented scripts rebuild the item files, after which `reproduce.py` recomputes Table 1 and `reproduce.py -rerun` reruns the simulations without an API key. The final Codabench submission is 1017682. The submission is two files that need only the Python standard library and NumPy.

Disclosures. Training data: public measurement-db only, for development, simulation and the eight calibration examples. No parameters are fitted to training responses. Pretrained models: Claude Opus 5.5 through the Anthropic API at test time, with Claude Sonnet 5.5 and Claude Haiku 5.5 as fallbacks. During development we also compared GPT and Gemini models. Development used LLM coding assistants (Claude Code and Codex).

References

[1] F. Maia Polo, L. Weber, L. Choshen, Y. Sun, G. Xu, M. Yurochkin. tinyBenchmarks: evaluating LLMs with fewer examples. arXiv:2402.14992, 2024.

[2] S. Truong, Y. Tu, P. Liang, B. Li, S. Koyejo. Reliable and efficient amortized model-based evaluation. arXiv:2503.13335, 2025.

[3] V. Hofmann, D. Heineman, I. Magnusson, K. Lo, et al. Fluid language model benchmarking. arXiv:2509.11106, 2025.

[4] R. Vivek, K. Ethayarajh, D. Yang, D. Kiela. Anchor Points: benchmarking models with much fewer examples. arXiv:2309.08638, 2023.

[5] G. Rasch. Probabilistic Models for Some Intelligence and Attainment Tests. Danish Institute for Educational Research, 1960.

[6] F. M. Lord. Applications of Item Response Theory to Practical Testing Problems. Erlbaum, 1980.

[7] W. J. van der Linden and C. A. W. Glas, editors. Elements of Adaptive Testing. Springer, 2010.

[8] R. J. Mislevy. Exploiting auxiliary information about examinees in the estimation of item parameters. Applied Psychological Measurement, 11(1):81–91, 1987.

[9] J. Park, E. Mendes, G. Stanovsky, A. Ritter. Anticipatory evaluation of language models: predicting LLM benchmark scores from descriptions. arXiv:2509.20645, 2025.

[10] Y. Zeng, D. Papailiopoulos. You don't need to run every eval. arXiv:2606.24020, 2026.

A Prior quality in detail

Table 1 uses the 71 pairs on which all three models gave a number, from 87 pairs on 25 public benchmarks, leaving out the eight pairs that serve as calibration examples. Every estimate saw five sample items. Opus gave no number on the eight pairs of two benchmarks, where the deployed predictor falls back to Sonnet and then Haiku. Table 4 scores that deployed chain on all 79 pairs, with the shrinkage chosen leave-one-benchmark-out and a 90% interval from a bootstrap over benchmarks, and two stricter splits.

Table 4: The deployed prior and stricter splits.

Prior	Pairs	RMSE	Correlation	Answers
Deployed: Opus, then Sonnet, then Haiku	79	0.153 [0.118, 0.184]	0.69	6 [3, 13]
Opus, calibration benchmarks removed	48	0.149	0.78	6
Opus, simulation benchmarks removed	35	0.131	0.62	10

B Benchmarks never used in development

We repeated the paired comparison of Section 5 on 15 benchmarks used in no development step, with item ratings off in both arms. Seven have mostly unnamed subjects, where the predictor leaves the prior out and the difference is zero by design. On the eight with mostly named subjects the prior changed the ALC by -0.0011 (90% interval -0.009 to $+0.007$ from a bootstrap over benchmarks). It helped on `aider` (-0.013), `bertaqa` (-0.021), `evouna` (-0.015), `healthadminbench` (-0.008) and `edu_circuit_hw` (-0.005). It hurt on `dbpa` ($+0.014$), whose subjects are prompt generators rather than language models, `ehrflowbench` ($+0.023$), with 20 items, and `clasheval` ($+0.005$), whose items open with the same 800 characters of instructions, so the prior's 600-character samples are identical.

C Pooling when subjects share a benchmark

This appendix tests the pooling machinery on its own, before the subject prior was added.

We simulated the protocol with per-pair 50/50 splits, streaming acquisition with the coverage rule of `labeling.py` applied to item IDs (this simulator forces a reveal against the full budget of 31 rather than the current checkpoint, which differs only near the end of a stream), nested budgets and labels pooled across pairs, with identical splits and streams for every method. These runs use the pooled predictor before the subject prior was added (v6), with item ratings from a cache of Claude Haiku ratings. We report three regimes: (i) *300 items*, 12 benchmarks with 300 sampled items each and up to 30 subjects, (ii) *online scale*, 100 items per benchmark with 2, 5 or 10 subjects, averaged over

Table 5: Brier ALC in simulation (lower is better), averaged over pairs. v5 pools benchmark and pair rates and adds item offsets. v6 adds the exact-item step. The official baseline is the empirical mean of the subject's own labels.

Regime	Baseline	v5	v6, random reveals	v6, coverage-first
300 items, 12 benchmarks	0.2412	0.1907	0.1846	**0.1835**
Online scale, 2 subjects	0.2561	0.2196	0.2192	**0.2170**
Online scale, 5 subjects	0.2310	0.1905	0.1890	**0.1886**
Online scale, 10 subjects	0.2368	0.1893	0.1858	**0.1829**
Full pools, no rating term	0.2494	0.2051	0.2036	**0.2027**

three seeds, and (iii) *full pools*, all items available for acquisition, evaluation capped at 150 items per pair and no rating term. ALC is averaged equally over pairs, as in the competition.

Table 5 shows that pooling does most of the work when subjects share a benchmark, from 0.2412 to 0.1907 on the 300-item regime. The exact-item step lowers the ALC by a further 0.006, and under random reveals it helps on all 12 benchmarks. Coverage-first acquisition lowers the ALC in 9 of 11 seed-level comparisons, by 0.0011 and 0.0031 on the two seeds of the 300-item regime. Removing the rating term raises the ALC from 0.1835 to 0.1871, with the largest gains on knowledge and code benchmarks such as `cmmlu` and `cruxeval` (Appendix D). Even with pooling, ability remains the main gap: replacing each subject's estimated rate with its true accuracy lowers the ALC from 0.1835 to 0.1709, about two thirds of it at budgets 1 and 3 (Figure 2). This oracle leaves budget 0 untouched, which the subject prior also addresses.

D Per-benchmark results

Table 6: ALC per benchmark on the 300-item regime. Subjects: number of subjects simulated on the benchmark.

Benchmark	Subjects	Baseline	v5	v6, coverage-first	no rating term
`agi_elo`	20	0.2244	0.1854	**0.1825**	0.1869
`cmmlu`	16	0.2615	0.1833	**0.1821**	0.2022
`ceval`	11	0.2954	0.2477	**0.2410**	0.2441
`cruxeval`	30	0.3151	0.2316	**0.2061**	0.2191
`bird_sql`	30	0.0792	0.0761	**0.0742**	0.0754
`harmbench`	29	0.2603	0.2196	0.2065	**0.2034**
`editbench`	30	0.3315	0.2401	0.2364	**0.2305**
`financebench`	16	0.2146	**0.1992**	0.2023	0.2027
`helm_afr`	22	0.3245	0.2548	**0.2495**	**0.2495**
`afrimedqa`	23	0.2442	0.1895	**0.1789**	0.1931
`agentdojo`	28	0.2206	0.1679	**0.1613**	0.1633
`benger`	24	0.1541	0.1327	0.1310	**0.1306**

E Online runs

F Variants that did not improve the submission

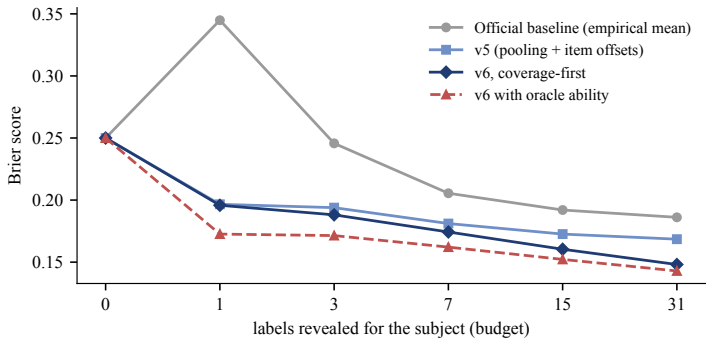


Figure 2: Brier score by budget on the 300-item regime. The oracle line replaces the estimated ability of each subject with its true accuracy on the benchmark.

Table 7: Every online formative run in Table 3. B_{31} is the mean Brier at budget 31, the draw's difficulty level.

Submission	Version	Pairs	Raw ALC	B_{31}	Excess
1016627	v6	9	0.158	0.116	0.042
1016628	v6	9	0.170	0.143	0.027
1016651	v6, no API key	8	0.158	0.134	0.024
1017498	+ subject prior (v19)	9	0.169	0.164	0.006
1017499	+ subject prior (v19)	9	0.159	0.151	0.008
1017545	+ subject prior (v23)	8	0.195	0.183	0.011
1017562	+ subject prior (v25)	8	0.179	0.169	0.010
1017629	+ subject prior (v27)	9	0.184	0.167	0.017
1017607	final	8	0.181	0.175	0.006
1017628	final	9	0.130	0.126	0.004
1017682	final (selected)	9	0.164	0.149	0.014

Table 8: Variants tried and dropped. Differences in pair-averaged Brier ALC against the predictor of the same stage (negative is better).

Variant	Result
Online Rasch model instead of shrinkage (v7, two seeds)	-0.0001 / $+0.0012$
Multi-dimensional item ratings (v8)	$+0.0010$ ($+0.004$ online scale)
Staged acquisition, ability first	$+0.0021$
Acquisition stratified by predicted difficulty	$+0.0013 \pm 0.0012$
Haiku answer agreement (v31, online)	adj. -0.011 to -0.014 vs -0.022
Opus and GPT prior averaged (simulation)	-0.0011 ± 0.0003
same, online (4 runs)	B_0 excess 0.048–0.085 vs 0.031–0.049
Text-learned difficulty (n-grams, embeddings, LoRA)	correlation ≤ 0.08
Prior ensembles (Opus twice, Opus with Sonnet)	RMSE 0.139–0.142 vs 0.141
Uncertainty sampling as in BLE (prior off)	$+0.0076 \pm 0.0015$
Gemini added to the Opus and GPT prior	RMSE 0.137 vs 0.138
GPT item ratings averaged with Opus	Brier 0.1391 vs 0.1387
Task extraction for ratings (`clasheval`)	rank correlation 0.005 vs 0.10